DeployMate

Zero-Downtime Deployment Orchestration Platform

Comprehensive Technical Project Report

Prepared: May 25, 2026

Submitted by

Reg. No: 12323849 | David Lakra

Reg. No: 12317981 | Alok Kumar Sharma

Faculty: Bharath Reddy

Project Name	DeployMate
Version	1.0.0
Document Type	Technical Project Report
Classification	Internal / Academic
Faculty Name	Bharath Reddy
Student 1	David Lakra
Registration No. 1	12323849
Student 2	Alok Kumar Sharma
Registration No. 2	12317981

Table of Contents

Github - https://github.com/lakradavid/Deploymate

1. Executive Summary

DeployMate is a full-stack, cloud-native deployment orchestration platform engineered to automate the complete software delivery lifecycle — from a developer's code push to a live, verified production deployment — with zero downtime. The platform addresses one of the most critical challenges in modern software engineering: reliable, repeatable, and observable deployments at scale.

The system is architected around three primary service layers: a React-based single-page application dashboard for real-time monitoring, an Express.js REST API backend for orchestration logic, and an asynchronous BullMQ worker service for executing the multi-stage deployment pipeline. MongoDB serves as the persistent data store for projects, deployments, and logs, while Redis underpins both the job queue and caching layer.

Key engineering achievements of DeployMate include a formally defined deployment state machine that enforces valid lifecycle transitions, a polling-based real-time UI that avoids loading flashes during background refreshes, a structured deployment logging subsystem, and a clean separation of concerns across controllers, services, and data models.

This report provides a comprehensive technical analysis of the DeployMate codebase, covering its architecture, data models, API design, frontend implementation, infrastructure configuration, CI/CD pipelines, security posture, and a candid assessment of the current implementation status alongside a prioritised roadmap for future development.

Key Metrics at a Glance

Metric	Value
Total Source Files	~35 files across 3 service layers
Frontend Pages	6 (Dashboard, Deployments, Details, Logs, Worker, Health)
Backend API Endpoints	4 (health, webhook, deployment trigger, status)
Database Models	4 (User, Project, Deployment, DeploymentLog)
Deployment Pipeline Stages	7 (Pending → Queued → Building → Deploying → Health Check → Success/Failed)
Frontend Polling Intervals	2s – 10s depending on page criticality
Primary Language	JavaScript (ES Modules + CommonJS)
UI Framework	React 19 + Vite 8
Backend Framework	Express 5 (latest)

2. Project Overview & Purpose

DeployMate was conceived to solve a fundamental problem faced by engineering teams of all sizes: the complexity and risk associated with deploying software to production. Manual deployments are error-prone, time-consuming, and often result in service downtime.

DeployMate automates this entire process, providing a reliable, auditable, and observable deployment pipeline.

2.1 Problem Statement

Traditional deployment workflows suffer from several critical pain points that DeployMate directly addresses:

- Manual deployments introduce human error and inconsistency across environments.
- Service downtime during updates degrades user experience and erodes trust.
- Lack of visibility into deployment progress makes debugging failures difficult.
- No automatic rollback means failed deployments require manual intervention.
- Scattered logs across multiple systems make post-mortem analysis challenging.

2.2 Solution Approach

DeployMate addresses these challenges through a webhook-driven, queue-backed deployment pipeline that decouples the trigger event from the execution, ensuring the API remains responsive while deployments run asynchronously in the background. The platform provides:

- Automated deployment triggering via GitHub push webhooks — no manual steps required.
- Rolling deployment strategy ensuring zero downtime during updates.
- Health-check validation before routing live traffic to new containers.
- Automatic rollback to the last stable version if health checks fail.
- Centralised, structured logging for every deployment event and stage transition.
- A real-time dashboard providing instant visibility into deployment status.

2.3 Target Users

DeployMate is designed for software engineering teams that deploy containerised applications and require a self-hosted, auditable deployment platform. It is particularly suited for:

- Small-to-medium engineering teams seeking CI/CD automation without vendor lock-in.
- DevOps engineers who need fine-grained control over the deployment pipeline.
- Development teams working with Docker-containerised Node.js or similar applications.
- Organisations requiring on-premise deployment orchestration for compliance reasons.

2.4 Core Feature Set

Feature	Description	Status
Webhook Trigger	GitHub push events automatically initiate deployments	Scaffolded
Job Queue	Redis-backed BullMQ queue decouples trigger from execution	Implemented
Docker Builds	Isolated, reproducible container builds per deployment	Planned
Rolling Deployments	Zero-downtime updates via parallel container strategy	Planned
Health Checks	Validates new instance before traffic switch	Simulated
Auto Rollback	Reverts to last stable version on failure	Planned
Deployment Logs	Structured per-event logging to MongoDB	Implemented
Real-Time Dashboard	React SPA with live polling and status visualisation	Implemented
JWT Authentication	Secure API access with token-based auth	Planned
Nginx Routing	Reverse proxy for traffic management	Planned

3. System Architecture

DeployMate employs a microservices-inspired, event-driven architecture that separates concerns across distinct service layers. This design ensures high availability of the API, fault isolation between components, and independent scalability of each service.

3.1 High-Level Architecture

The system is composed of four primary layers that communicate through well-defined interfaces:

Layer	Technology	Responsibility
Presentation Layer	React 19 SPA (Vite)	User interface, real-time monitoring dashboard
API Layer	Express.js 5 (Node.js)	REST endpoints, webhook reception, orchestration
Worker Layer	BullMQ + Redis	Async job processing, deployment pipeline execution
Data Layer	MongoDB (Mongoose)	Persistent storage for projects, deployments, logs
Infrastructure Layer	Docker, Nginx, Redis	Container orchestration, routing, caching/queuing

3.2 Request Flow

A typical deployment follows this sequence of events:

- Step 1 — TRIGGER: Developer pushes code to the main branch on GitHub.
- Step 2 — WEBHOOK: GitHub fires a POST request to the DeployMate webhook endpoint (/api/webhook).
- Step 3 — VALIDATION: The backend validates the webhook signature using HMAC-SHA256.
- Step 4 — QUEUE: A new deployment job is created and pushed to the Redis-backed BullMQ queue.
- Step 5 — WORKER PICKUP: The BullMQ worker consumes the job and begins the pipeline.
- Step 6 — BUILD: The worker builds a new Docker image from the repository source.
- Step 7 — DEPLOY: A new container is started in parallel with the currently running instance.
- Step 8 — HEALTH CHECK: The worker polls the new container's health endpoint until it responds.
- Step 9 — TRAFFIC SWITCH: Nginx configuration is updated to route traffic to the new container.
- Step 10 — CLEANUP: The old container is gracefully terminated, completing the zero-downtime cycle.

3.3 Project Directory Structure

```
Deploymate-main/
├── frontend/                  # React SPA dashboard (Vite build tool)
│   ├── src/
```

```
│   │   ├── pages/              # Route-level page components (6 pages)
│   │   ├── components/         # Reusable UI components
│   │   ├── hooks/              # Custom React hooks (usePolling)
│   │   └── api/                # API client abstraction layer
├── backend/                    # Primary Express API + Mongoose models
│   ├── Models/                 # User, Project, Deployment schemas
│   ├── controllers/            # Request handlers
│   ├── routes/                 # Express route definitions
│   ├── services/               # Business logic layer
│   └── workers/                # BullMQ deployment worker
├── api/                        # Secondary API scaffold (CommonJS)
│   └── src/
│       ├── controllers/        # Request handlers
│       ├── models/             # DeploymentLog schema
│       ├── services/           # Service stubs
│       └── utils/              # Deployment state machine
├── .github/workflows/          # GitHub Actions CI/CD pipelines
├── docker-compose.yml          # Infrastructure orchestration (planned)
└── Dockerfile                  # Container build definition (planned)
```

3.4 Inter-Service Communication

The backend API and worker service communicate exclusively through the Redis job queue, providing loose coupling and resilience. If the worker crashes, jobs remain in the queue and are retried automatically by BullMQ. The frontend communicates with the backend via HTTP REST calls using the native Fetch API, with a polling strategy for near-real-time updates.

4. Technology Stack

DeployMate leverages a modern, production-grade technology stack carefully selected for performance, developer experience, and ecosystem maturity. Each technology choice reflects deliberate engineering decisions aligned with the platform's requirements.

4.1 Frontend Technologies

Package	Version	Purpose	Rationale
React	^19.2.4	UI framework	Latest stable React with concurrent features
react-dom	^19.2.4	DOM rendering	Paired with React core
react-router-dom	^7.15.1	Client-side routing	Declarative routing for SPA navigation
lucide-react	^1.16.0	Icon library	Lightweight, consistent SVG icon set

Vite	^8.0.1	Build tool / dev server	Extremely fast HMR and optimised production builds
ESLint	^9.39.4	Code linting	Enforces code quality and consistency

4.2 Backend Technologies

Package	Version	Purpose	Rationale
Node.js	v18+	Runtime environment	Non-blocking I/O ideal for async deployment orchestration
Express.js	^5.2.1	HTTP server framework	Express 5 with improved async error handling
Mongoose	^9.6.0	MongoDB ODM	Schema validation and query abstraction for MongoDB
Redis	^5.12.1	Cache & queue backing	High-performance in-memory store for BullMQ
BullMQ	latest	Job queue processor	Robust Redis-backed queue with retry and concurrency
dotenv	^17.4.2	Environment config	Secure injection of environment variables

4.3 Infrastructure Technologies

Technology	Role	Notes
MongoDB	Primary database	Document store for users, projects, deployments, logs
Redis	Queue + cache	BullMQ job queue backing store and session cache
Docker	Containerisation	Isolated, reproducible build and runtime environments
Docker Compose	Local orchestration	Multi-service local development environment
Nginx	Reverse proxy / LB	Traffic routing, SSL termination, rolling deploy switch
AWS EC2	Cloud compute	Target deployment environment for production
GitHub Actions	CI/CD automation	Automated build, test, and deployment pipelines

4.4 Module System Notes

An important architectural observation is that the two backend service folders use different JavaScript module systems:

- backend/ — Uses ES Modules ("type": "module" in package.json). Uses import/export syntax throughout.
- api/ — Uses CommonJS (require/module.exports). However, deploymentLogModel.js incorrectly uses ES Module syntax, which will cause a runtime error.

This inconsistency is a known technical debt item that must be resolved before the api/ service can be run in production. The recommended resolution is to standardise both services on ES Modules.

5. Database Design & Data Models

DeployMate uses MongoDB as its primary database, accessed through the Mongoose ODM. The schema design follows a document-oriented approach with references (ObjectId) between related collections, balancing query performance with data normalisation.

5.1 Database Connection

The database connection is managed in backend/Models/db.js using Mongoose's connect() method. The connection URI is read from the MONGO_URI environment variable, defaulting to mongodb://localhost:27017/deploymate for local development. The application exits with a non-zero code if the connection fails, preventing the server from starting in a broken state.

5.2 User Model

Collection: users | File: backend/Models/User.js

Field	Type	Constraints	Description
name	String	required	Full name of the user
email	String	required, unique	Email address used for login
password	String	required	Hashed password (bcrypt recommended)
githubId	String	default: null	GitHub OAuth ID for social login
createdAt	Date	auto (timestamps)	Record creation timestamp
updatedAt	Date	auto (timestamps)	Record last-updated timestamp

5.3 Project Model

Collection: projects | File: backend/Models/Project.js

Field	Type	Constraints	Description
userId	ObjectId	required, ref: User	Owner of the project
name	String	required	Human-readable project name
githubRepoUrl	String	required	Full GitHub repository URL
branch	String	default: "main"	Branch to deploy from
port	Number	required	Port the container exposes
envVars	[{k,v}]	array of key-value pairs	Environment variables injected at runtime
webhookSecret	String	required	HMAC secret for webhook validation
status	Enum	active | inactive | failed	Current project health status

5.4 Deployment Model

Collection: deployments | File: backend/Models/Deployment.js

Field	Type	Constraints	Description
projectId	ObjectId	required, ref: Project	Associated project
commitHash	String	required	Git commit SHA that triggered deployment
commitMessage	String	optional	Commit message for display

status	Enum	pending|building|deploying|success|failed	Current deployment lifecycle state
logs	[String]	array	Inline log messages (legacy field)
startTime	Date	default: Date.now	When the deployment began
endTime	Date	optional	When the deployment completed

5.5 DeploymentLog Model

Collection: deploymentLogs | File: api/src/models/deploymentLogModel.js

Field	Type	Constraints	Description
deploymentId	String	required, indexed	References the deployment (string ID)
message	String	required	Human-readable log message
status	Enum	INFO|SUCCESS|FAILED|ERROR|WARN|PENDING|BUILDING	Severity/type of the log entry
timestamp	Date	default: Date.now	When the event occurred

Note: The deploymentId field in DeploymentLog is a String rather than an ObjectId reference. This allows logs to be created before a MongoDB deployment document exists, but means referential integrity is not enforced at the database level.

6. Backend API – Routes & Controllers

The backend API follows the MVC (Model-View-Controller) pattern, with a clean separation between route definitions, controller logic, and service-layer business rules. The server runs on Express.js 5, which provides improved async/await error propagation compared to Express 4.

6.1 Server Initialisation

The entry point (backend/index.js) performs the following startup sequence:

- 1. Creates an Express application instance.
- 2. Establishes a Redis connection using the redis client library.
- 3. Connects to MongoDB via the connectDB() utility function.
- 4. Registers JSON body-parsing middleware.
- 5. Mounts route handlers under the /api prefix.
- 6. Starts the HTTP server on port 3000.

6.2 API Endpoint Reference

Base URL: http://localhost:3000/api

Method	Endpoint	Controller	Description	Response
GET	/api/health	healthController.getHealth	System health check	200 { status, timestamp, uptime }
POST	/api/webhook	webhookController.handleWebhook	Receive GitHub webhook event	200 { success, message }
POST	/api/deployment	deploymentController.triggerDeployment	Trigger manual deployment	202 { success, deploymentId }
GET	/api/deployment/:id/status	deploymentController.getStatus	Get deployment status by ID	200 { deploymentId, status }

6.3 Controller Implementation Details

Health Controller

The health controller (backend/controllers/healthController.js) delegates to the health service and returns a JSON response. It wraps the service call in a try/catch block, returning a 500 status with an "unhealthy" status if the service throws. This provides a reliable liveness probe endpoint for container orchestration systems.

Webhook Controller

The webhook controller (backend/controllers/webhookController.js) accepts the raw request body and passes it to the webhook service for processing. In the current implementation, the service logs the payload and returns a success response. The intended production behaviour is to validate the GitHub HMAC-SHA256 signature and push a job to the Redis queue.

Deployment Controller

The deployment controller (backend/controllers/deploymentController.js) exposes two operations: triggering a new deployment (POST) and querying deployment status (GET). The trigger endpoint returns HTTP 202 Accepted, correctly signalling that the request has been received but processing is asynchronous. The status endpoint returns the current state of a deployment by ID.

6.4 Secondary API Service (api/)

The api/ directory contains an earlier scaffold of the API service using CommonJS modules and Express 4. It exposes three routes:

Method	Endpoint	Controller	Description
GET	/health	healthController.checkHealth	Returns { status: "OK" }
POST	/webhook	webhookController.handleWebhook	Accepts webhook, returns 202
POST	/deploy	deployController.triggerDeploy	Triggers deployment, returns

			202

This service appears to be a prototype that predates the backend/ service. The backend/ service is the active, primary API. The api/ service should either be merged into backend/ or removed to avoid confusion.

7. Business Logic & Services

The service layer encapsulates all business logic, keeping controllers thin and focused on HTTP concerns. Each service module exports pure async functions that can be independently tested and reused across different controllers or workers.

7.1 Health Service

File: backend/services/healthService.js | Status: Functional

The health service provides a checkSystemHealth() function that returns the current system status. In the current implementation it returns a static "healthy" status with the process uptime. The production implementation should extend this to:

- Check MongoDB connection state via mongoose.connection.readyState.
- Ping the Redis server to verify queue availability.
- Report the number of active BullMQ workers and pending jobs.
- Return an "unhealthy" status if any critical dependency is unavailable.

7.2 Webhook Service

File: backend/services/webhookService.js | Status: Stub

The webhook service currently logs the incoming payload and returns a success response. The complete implementation requires:

- HMAC-SHA256 signature validation using the project's webhookSecret from MongoDB.
- Parsing the GitHub push event payload to extract repository URL, branch, and commit hash.
- Looking up the corresponding Project document in MongoDB.
- Creating a new Deployment document with status "pending".
- Pushing a job to the BullMQ "deployment-queue" with the deployment ID and project config.

7.3 Deployment Service

File: backend/services/deploymentService.js | Status: Stub

The deployment service exposes two functions. triggerDeployment() currently returns a hardcoded mock response with deploymentId "dep-123". getDeploymentStatus() returns a

hardcoded "IN_PROGRESS" status. Both functions require full implementation to interact with MongoDB and the BullMQ queue.

7.4 Deployment Log Service

File: api/src/services/deploymentLogService.js | Status: Functional

This is one of the most complete service implementations in the codebase. The createLog() function accepts a deploymentId, message, and optional status (defaulting to "INFO"), creates a DeploymentLog document, and persists it to MongoDB. It includes proper error handling with a descriptive error message if the save operation fails.

```
createLog(deploymentId, message, status = "INFO") -> Promise<DeploymentLog>
```

This service is called by the BullMQ worker at each stage of the deployment pipeline to create an auditable trail of events.

7.5 Service Layer Design Principles

The service layer follows several important design principles that should be maintained as the implementation matures:

Principle	Implementation	Benefit
Single Responsibility	Each service handles one domain concern	Easier testing and maintenance
Async/Await	All service functions are async	Clean error propagation with try/catch
Error Propagation	Services throw errors; controllers catch them	Consistent HTTP error responses
No HTTP Coupling	Services have no knowledge of req/res objects	Services are reusable outside HTTP context
Dependency Injection	Connection configs passed as parameters to worker	Testable without live infrastructure

8. Deployment State Machine

One of the most architecturally significant components of DeployMate is the formal deployment state machine defined in api/src/utils/deploymentState.js. This module enforces valid lifecycle transitions, preventing deployments from entering invalid states and providing a single source of truth for the deployment lifecycle.

8.1 State Definitions

State	Description	Terminal?	Colour in UI
PENDING	Deployment created, awaiting queue slot	No	Grey
QUEUED	Job added to BullMQ queue, awaiting worker pickup	No	Grey
BUILDING	Worker is building the Docker image	No	Amber (#F59E0B)

DEPLOYING	New container is starting up	No	Blue (#3B82F6)
HEALTH_CHECKING	Worker is polling the new container's health endpoint	No	Purple (#8B5CF6)
SUCCESS	Deployment completed, traffic switched to new container	Yes	Green (#10B981)
FAILED	Deployment failed at any stage, rollback initiated	Yes	Red (#EF4444)

8.2 Valid State Transitions

The state machine enforces a strict linear progression with a single branching point at HEALTH_CHECKING. The allowed transitions are:

```
PENDING         → QUEUED
QUEUED          → BUILDING
BUILDING        → DEPLOYING
DEPLOYING       → HEALTH_CHECKING
HEALTH_CHECKING → SUCCESS   (health check passed)
HEALTH_CHECKING → FAILED    (health check failed or timeout)
SUCCESS         → (terminal — no further transitions)
FAILED          → (terminal — no further transitions)
```

8.3 API Functions

The state machine module exports two functions:

isValidTransition(currentState, nextState)

Returns a boolean indicating whether the transition from currentState to nextState is permitted. Returns false if currentState is not a recognised state. This function is safe to call without side effects and is suitable for validation before attempting a transition.

transitionState(currentState, nextState)

Attempts to transition to nextState. If the transition is valid, returns the new state string. If the transition is invalid, throws an Error with a descriptive message including both states. This fail-fast behaviour ensures that invalid state transitions are caught immediately rather than silently corrupting deployment records.

8.4 Design Significance

The formal state machine provides several important guarantees for the system:

- Prevents a deployment from jumping from PENDING directly to SUCCESS, bypassing build and health checks.
- Ensures terminal states (SUCCESS, FAILED) cannot be overwritten by late-arriving worker messages.
- Provides a clear contract for the worker service — it knows exactly which transitions are valid at each stage.
- Enables the frontend to display accurate, meaningful status information to users.

- Creates an auditable trail when combined with the deployment log service.

8.5 Note on State Discrepancy

There is a minor inconsistency between the state machine (7 states including QUEUED and HEALTH_CHECKING) and the Deployment MongoDB model (5 states: pending, building, deploying, success, failed). The QUEUED and HEALTH_CHECKING states exist in the state machine but not in the database schema. This should be reconciled by updating the Deployment model's status enum to include all 7 states.

9. BullMQ Worker & Job Queue

The BullMQ worker is the execution engine of DeployMate. It consumes deployment jobs from the Redis-backed queue and orchestrates the multi-stage deployment pipeline. BullMQ was chosen for its robust retry mechanisms, job prioritisation, concurrency control, and comprehensive event system.

9.1 Worker Architecture

The worker is defined in backend/workers/deploymentWorker.js and exported as a factory function setupDeploymentWorker(connectionConfig). This design allows the Redis connection configuration to be injected, making the worker testable without a live Redis instance.

9.2 Pipeline Execution Stages

When a job is consumed from the "deployment-queue", the worker executes the following sequential stages:

Stage	Action	State Transition	Log Message
1. Job Receipt	Log job received	PENDING → QUEUED	"deployment received"
2. Build Start	Update status, log build start	QUEUED → BUILDING	"build started"
3. Build App	Execute buildApplication(jobData)	BUILDING (in progress)	Simulated 1s delay
4. Deploy	Execute deployContainer(jobData)	BUILDING → DEPLOYING	Simulated 1s delay
5. Health Check	Execute healthCheck(jobData)	DEPLOYING → HEALTH_CHECKING	Simulated 1s delay
6. Finalise	Update status to SUCCESS	HEALTH_CHECKING → SUCCESS	"deployment complete"

9.3 Error Handling

The worker registers two BullMQ event listeners for robust error handling:

- completed event: Logs a success message with the job ID when a deployment finishes successfully.

- failed event: Updates the deployment status to FAILED in the database and creates a failure log entry with the error message. The job object is null-checked before accessing its ID to handle edge cases.

9.4 Simulated vs Production Stages

The current worker implementation uses simulated stages with 1-second delays to represent the build, deploy, and health check operations. The production implementation of each stage would involve:

Stage Function	Current	Production Implementation
buildApplication()	1s simulated delay	docker build -t <image>:<tag> . in the cloned repo directory
deployContainer()	1s simulated delay	docker run -d -p <port>:<port> --name <container> <image>:<tag>
healthCheck()	1s simulated delay	HTTP GET to container health endpoint with retry and timeout logic

9.5 BullMQ Advantages

BullMQ provides several production-grade features that make it ideal for deployment orchestration:

- Automatic job retry with configurable backoff strategies (exponential, fixed).
- Job prioritisation to ensure critical deployments are processed first.
- Concurrency control to limit simultaneous deployments and prevent resource exhaustion.
- Job persistence in Redis — jobs survive worker restarts without being lost.
- Delayed jobs for scheduling deployments at specific times.
- Job progress reporting for granular status updates during long-running stages.
- Dead letter queue for jobs that exhaust all retry attempts.

10. Frontend Application

The DeployMate frontend is a React 19 single-page application built with Vite 8. It provides a real-time deployment monitoring dashboard with six distinct views, a consistent dark-themed design system, and a custom polling hook for live data updates.

10.1 Application Routing

The application uses React Router v7 with a nested route structure. All routes are wrapped in the AppLayout component, which provides the persistent sidebar and top bar. Unknown routes redirect to the root path.

Route	Component	Description
/	Dashboard	Overview with stats cards and recent deployments table

/deployments	DeploymentList	Full paginated list of all deployments
/deployment/:id	DeploymentDetails	Detailed view of a single deployment with live logs
/deployment/:id/logs	DeploymentLogsViewer	Full-screen log viewer for a deployment
/worker	WorkerStatusPanel	BullMQ queue statistics and worker health
/health	SystemHealthOverview	API server, MongoDB, and Redis status cards

10.2 API Client

The frontend uses a lightweight API client abstraction (frontend/src/api/client.js) built on the native Fetch API. The client exposes get() and post() methods that:

- Prepend the base URL (http://localhost:3000/api) to all endpoint paths.
- Set the Content-Type: application/json header for POST requests.
- Check response.ok and throw a descriptive Error if the status is not 2xx.
- Parse and return the JSON response body.

This abstraction keeps API call logic out of components and makes it easy to swap the underlying HTTP library or add authentication headers in a single place.

10.3 Page Implementations

Dashboard

The Dashboard page polls GET /deployments?limit=5 every 5 seconds. It displays three summary stat cards (Total Deployments, Success Rate, Active Workers) and a recent deployments table with ID, project, status badge, and date columns. The success rate (98.5%) and active workers (3) are currently hardcoded placeholders.

DeploymentList

Polls GET /deployments every 5 seconds to display the complete deployment history. Includes pagination support via meta.total from the API response.

DeploymentDetails

Polls GET /deployment/:id every 3 seconds for a specific deployment. Displays full metadata and embeds the LiveLogStream component for real-time log viewing.

DeploymentLogsViewer

Polls GET /logs/:id every 2 seconds — the fastest polling interval in the application, reflecting the high update frequency of active deployment logs.

WorkerStatusPanel

Polls GET /health every 10 seconds to display BullMQ queue statistics. Currently renders placeholder data as the health endpoint does not yet return queue metrics.

SystemHealthOverview

Polls GET /health every 10 seconds and displays three status cards for the API server (with uptime), MongoDB connection, and Redis connection. Uses lucide-react icons (Activity, Database, Server) for visual differentiation.

11. Frontend Components & UI Design

The frontend component library is purpose-built for the DeployMate dashboard. All components use plain CSS with custom properties (CSS variables) rather than a CSS framework, giving precise control over the dark-themed design system.

11.1 Component Inventory

Component	File	Description
AppLayout	components/AppLayout.jsx	Root layout: renders Sidebar + Outlet for page content
Sidebar	components/Sidebar.jsx	Left navigation with NavLink items and lucide icons
Layout	components/Layout/Layout.jsx	Alternative layout with search bar and user avatar (legacy)
DeploymentCard	components/DeploymentCard/DeploymentCard.jsx	Card showing repo, branch, commit, version with inline log toggle
StatusBadge	components/StatusBadge/StatusBadge.jsx	Pill badge with coloured dot; animated pulse for active states
LiveLogStream	components/LiveLogStream.jsx	Auto-scrolling log viewer with timestamp and message columns
SkeletonLoader	components/SkeletonLoader.jsx	Shimmer loading placeholder with configurable rows and height
EmptyState	components/EmptyState.jsx	Centred empty message with Archive icon from lucide-react
ErrorState	components/ErrorState.jsx	Error display with retry button callback

11.2 Design System

The design system is defined entirely through CSS custom properties in frontend/src/index.css. This approach provides a single source of truth for all visual tokens:

Colour Palette

Token	Value	Usage
--bg-primary	#0a0a0a	Main page background
--bg-secondary	#141414	Card and sidebar backgrounds
--bg-tertiary	#1f1f1f	Hover states, code blocks
--text-primary	#ededed	Primary text
--text-secondary	#a1a1aa	Labels, subtitles

--status-success	#10b981	Success state (green)
--status-failed	#ef4444	Failed state (red)
--status-building	#f59e0b	Building state (amber)
--status-deploying	#3b82f6	Deploying state (blue)
--status-health	#8b5cf6	Health checking state (purple)

StatusBadge Component

The StatusBadge component is a key UI element that provides immediate visual feedback on deployment state. Each status maps to a unique colour combination for background, text, and border. The BUILDING and DEPLOYING states include a CSS pulse animation on the status dot to indicate active processing.

LiveLogStream Component

The LiveLogStream component renders an auto-scrolling log viewer. It uses a useRef to access the scroll container and a useEffect that triggers scrollTop = scrollHeight whenever the logs array changes. Each log entry displays a formatted timestamp (HH:MM:SS.mmm) and the message text in a monospace font. The component gracefully handles empty log arrays with a "No logs available" placeholder.

SkeletonLoader Component

The SkeletonLoader provides a shimmer loading animation using a CSS gradient animation. It renders configurable rows of placeholder content, preventing layout shift when data loads. The shimmer effect uses a 1000px wide gradient that animates across the element over 2 seconds, creating a smooth scanning effect.

11.3 Typography

The application uses the Inter typeface loaded from Google Fonts, with a fallback stack of -apple-system, BlinkMacSystemFont, Segoe UI, Roboto, Helvetica, Arial. Inter was chosen for its excellent legibility at small sizes and its tabular number variant, which ensures deployment IDs and timestamps align correctly in tables.

12. Real-Time Data & Polling Strategy

DeployMate's frontend achieves near-real-time data updates through a carefully engineered polling strategy implemented in the custom usePolling hook. This approach was chosen over WebSockets for its simplicity and compatibility with standard REST APIs, while still providing a responsive user experience.

12.1 The usePolling Hook

File: frontend/src/hooks/usePolling.js

The usePolling hook accepts a fetch function and a polling interval (default 5000ms). It returns { data, loading, error, refetch } — a standard data-fetching interface familiar to React developers.

Key Engineering Decisions

Decision	Implementation	Benefit
Stable fetch function reference	useRef stores the latest fetchFn; useEffect updates the ref without restarting the poll loop	Prevents infinite re-render loops when fetchFn is defined inline in a component
No loading flash on background polls	isBackground parameter skips setLoading(true) for polls after the initial fetch	UI does not flicker on every poll interval — only shows spinner on first load
Stable fetchData callback	useCallback with empty dependency array creates a stable function reference	The polling useEffect only runs once, not on every render
Cleanup on unmount	isMounted flag and clearTimeout(timeoutId) in the effect cleanup	Prevents setState calls on unmounted components and memory leaks
Manual refetch	refetch() calls fetchData(false) to show loading spinner on demand	Users can force a refresh without waiting for the next poll interval

12.2 Polling Intervals by Page

Different pages use different polling intervals based on the expected update frequency and the criticality of the data:

Page	Interval	Rationale
DeploymentLogsViewer	2 seconds	Logs update frequently during active deployments; fast polling is critical
DeploymentDetails	3 seconds	Status changes are important but slightly less frequent than log lines
Dashboard	5 seconds	Overview data; moderate freshness acceptable
DeploymentList	5 seconds	List view; same as dashboard
SystemHealthOverview	10 seconds	Infrastructure health changes slowly; aggressive polling wastes resources
WorkerStatusPanel	10 seconds	Queue stats change slowly; conservative interval appropriate

12.3 Comparison with WebSockets

The README roadmap mentions WebSockets as a future enhancement. The trade-offs between the current polling approach and WebSockets are:

Aspect	Polling (Current)	WebSockets (Planned)
Latency	2-10 seconds depending on interval	Sub-second, event-driven
Server Load	Predictable, proportional to clients	Lower per-update, but persistent connections
Implementation	Simple — works with any REST API	Requires server-side socket management
Reliability	Self-healing — next poll recovers	Requires reconnection logic
Scalability	Stateless — easy to load balance	Stateful connections complicate load balancing

For the current scale of DeployMate, polling is the pragmatic choice. WebSockets would become beneficial when the platform serves many concurrent users monitoring active deployments.

13. Infrastructure & DevOps Configuration

DeployMate's infrastructure is designed around Docker containerisation with Nginx as the reverse proxy and Redis as the shared communication backbone. While the configuration files are currently empty placeholders, the README and codebase provide a clear picture of the intended production architecture.

13.1 Docker Architecture

The platform is designed to run as a set of Docker containers orchestrated by Docker Compose. The intended service topology is:

Service	Image	Port	Role
frontend	node:18-alpine (Vite build)	5173 → 80	React SPA served by Nginx or Node
backend	node:18-alpine	3000	Express API server
worker	node:18-alpine	N/A	BullMQ worker process (no HTTP port)
mongodb	mongo:7	27017	Primary database
redis	redis:7-alpine	6379	Job queue and cache
nginx	nginx:alpine	80, 443	Reverse proxy, SSL termination, traffic routing

13.2 Environment Variables

The application is configured entirely through environment variables, following the Twelve-Factor App methodology. The required variables are:

Variable	Service	Description	Example
PORT	backend	HTTP server port	3000
MONGO_URI	backend	MongoDB connection string	mongodb://mongo:27017/deploymate
REDIS_HOST	backend	Redis server hostname	redis
REDIS_PORT	backend	Redis server port	6379
JWT_SECRET	backend	Secret for JWT token signing	a-long-random-string
WEBHOOK_SECRET	backend	GitHub webhook HMAC secret	github-webhook-secret

13.3 Nginx Configuration

Nginx serves two critical roles in the DeployMate architecture:

- Reverse Proxy: Routes incoming HTTP requests to the appropriate backend service based on path prefix (/api → backend, / → frontend).
- Traffic Switch: During a rolling deployment, Nginx's upstream configuration is updated to point to the new container, enabling zero-downtime traffic switching.

The zero-downtime switch works by maintaining two upstream server definitions (blue and green) and atomically updating the active upstream when a new deployment passes health checks. Nginx's graceful reload (nginx -s reload) applies the new configuration without dropping existing connections.

13.4 AWS EC2 Deployment

The production deployment target is AWS EC2. The recommended setup involves:

- An EC2 instance (t3.medium or larger) running Amazon Linux 2 or Ubuntu 22.04.
- Docker and Docker Compose installed on the instance.
- An Elastic IP address for stable DNS resolution.
- Security groups allowing inbound traffic on ports 80 (HTTP) and 443 (HTTPS).
- An IAM role with permissions to pull from ECR (if using private Docker images).
- GitHub Actions deploying to the EC2 instance via SSH on push to main.

14. CI/CD Pipeline & GitHub Actions

DeployMate includes three GitHub Actions workflow files in the .github/workflows/ directory. These workflows define the automated CI/CD pipeline that builds, tests, and deploys the application on code changes.

14.1 Workflow Overview

Workflow File	Trigger	Status	Purpose
deploy.yaml	Push to main	Incomplete (no steps)	Deploy to Netlify
npm-publish-github-packages.yml	GitHub Release created	Functional	Build, test, publish to GitHub Packages
release.yml	GitHub Release created	Functional (duplicate)	Identical to npm-publish workflow

14.2 deploy.yaml

This workflow is intended to deploy the application to Netlify when code is pushed to the main branch. Currently, the workflow defines the trigger and a single job (deploy) running on ubuntu-latest, but contains no steps. The complete implementation would include:

- Checkout the repository with actions/checkout@v4.
- Set up Node.js with actions/setup-node@v4.
- Install frontend dependencies with npm ci.
- Build the frontend with npm run build.
- Deploy the dist/ directory to Netlify using the Netlify CLI or netlify-actions.

14.3 npm-publish-github-packages.yml

This workflow triggers on GitHub Release creation and performs two jobs:

Job 1: build

- Checks out the repository.
- Sets up Node.js version 20.
- Runs npm ci for a clean, reproducible dependency install.
- Runs npm test to validate the build.

Job 2: publish-gpr

- Depends on the build job completing successfully.
- Sets up Node.js with the GitHub Packages registry URL.
- Runs npm ci.
- Publishes the package to GitHub Packages using the GITHUB_TOKEN secret.

14.4 Recommended CI/CD Improvements

To achieve a production-grade CI/CD pipeline, the following enhancements are recommended:

Enhancement	Priority	Description
Complete deploy.yaml	High	Add all deployment steps including build and Netlify/EC2 deploy
Add backend tests	High	Implement Jest/Vitest tests and run them in CI
Docker image build in CI	High	Build and push Docker image to ECR/GHCR on each release
Environment secrets	High	Store MONGO_URI, JWT_SECRET, etc. as GitHub Actions secrets
Remove duplicate workflow	Medium	Merge release.yml into npm-publish-github-packages.yml
Add staging environment	Medium	Deploy to staging on PR merge, production on release
Security scanning	Medium	Add npm audit and Snyk/Dependabot for vulnerability scanning
Lint and format checks	Low	Run ESLint in CI to enforce code quality on every PR

15. Security Considerations

Security is a critical concern for a deployment orchestration platform, as it has direct access to production infrastructure. This section analyses the current security posture and provides recommendations for hardening the platform before production deployment.

15.1 Current Security Implementations

Security Control	Status	Details
Webhook signature validation	Planned	webhookSecret stored in Project model; HMAC validation not yet implemented

JWT authentication	Planned	JWT_SECRET in env vars; auth middleware not yet implemented
Environment variable secrets	Implemented	Sensitive config loaded from .env via dotenv; not hardcoded
Input validation	Partial	Mongoose schema validation provides basic type checking
HTTPS/TLS	Planned	Nginx intended to handle SSL termination
Password hashing	Not started	User model stores password field but no bcrypt hashing implemented

15.2 Critical Security Gaps

Webhook Signature Validation

The webhook endpoint currently accepts any POST request without validating the GitHub HMAC-SHA256 signature. This means any actor who discovers the webhook URL can trigger arbitrary deployments. This is a critical vulnerability that must be fixed before exposing the endpoint to the internet. The fix requires computing HMAC-SHA256 of the raw request body using the project's webhookSecret and comparing it to the X-Hub-Signature-256 header using a timing-safe comparison (crypto.timingSafeEqual).

Missing Authentication Middleware

All API endpoints are currently unauthenticated. Any client with network access can trigger deployments, query deployment status, or access health information. JWT authentication middleware must be implemented and applied to all sensitive routes before production deployment.

Password Storage

The User model includes a password field but there is no evidence of bcrypt hashing in the codebase. Passwords must never be stored in plaintext. The implementation must use bcrypt (cost factor 12+) or Argon2 to hash passwords before storage.

Debug Code in Production

The backend/index.js file contains a hardcoded Redis JSON set operation for a test user (user:1 with Alice's data). This debug code must be removed before production deployment as it writes test data to the production Redis instance on every server start.

15.3 Security Recommendations

Recommendation	Priority	Implementation
Implement webhook HMAC validation	Critical	Use crypto.createHmac("sha256", secret).update(rawBody).digest("hex")
Add JWT auth middleware	Critical	Use jsonwebtoken library; protect all non-health routes
Hash passwords with bcrypt	Critical	bcrypt.hash(password, 12) on registration; bcrypt.compare on login
Remove debug Redis code	High	Delete the client.json.set("user:1"...) block from index.js
Add rate limiting	High	Use express-rate-limit on webhook and auth endpoints
Implement CORS policy	High	Restrict allowed origins to the frontend domain
Add Helmet.js	Medium	Set security headers (CSP, HSTS, X-Frame-Options, etc.)
Sanitise log output	Medium	Ensure deployment logs do not capture environment

		variable values
Rotate webhook secrets	Medium	Provide an API endpoint to regenerate webhookSecret per project

16. Current Implementation Status & Gaps

This section provides an honest, comprehensive assessment of what has been implemented, what is stubbed or partially complete, and what is entirely absent. This analysis is essential for planning the next development sprint.

16.1 Fully Implemented Components

Component	Location	Notes
React SPA with routing	frontend/src/	All 6 pages, components, and hooks complete
Dark theme design system	frontend/src/index.css	Full CSS variable system with status colours
usePolling custom hook	frontend/src/hooks/usePolling.js	Production-quality with stable refs and cleanup
API client abstraction	frontend/src/api/client.js	GET and POST with error handling
MongoDB models	backend/Models/	User, Project, Deployment schemas complete
Database connection	backend/Models/db.js	With error handling and env var config
Deployment state machine	api/src/utils/deploymentState.js	All 7 states and valid transitions defined
DeploymentLog model + service	api/src/models/ + services/	Schema and createLog() function complete
BullMQ worker scaffold	backend/workers/deploymentWorker.js	Pipeline structure with simulated stages
Health service	backend/services/healthService.js	Returns uptime and healthy status
Express route structure	backend/routes/	All three route files properly structured
Controller layer	backend/controllers/	All controllers with try/catch error handling

16.2 Stubbed / Partially Implemented

Component	Gap	Effort to Complete
webhookService.processWebhook()	No HMAC validation or Redis queue push	Medium
deploymentService.triggerDeployment()	Returns hardcoded mock data	Medium
deploymentService.getDeploymentStatus()	Returns hardcoded "IN_PROGRESS"	Low
Worker buildApplication()	Simulated 1s delay instead of docker build	High
Worker deployContainer()	Simulated 1s delay instead of docker run	High
Worker healthCheck()	Simulated 1s delay instead of HTTP polling	Medium
Health service dependencies	Does not check MongoDB or Redis connection state	Low
Dashboard stats	Success rate and active workers are hardcoded	Medium

GitHub Actions deploy.yaml	Job defined but has no steps	Low

16.3 Missing / Not Implemented

Feature	Impact	Priority
JWT authentication middleware	All endpoints are unauthenticated	Critical
Webhook HMAC signature validation	Webhook endpoint is open to abuse	Critical
Password hashing (bcrypt)	Passwords stored insecurely	Critical
Docker/Compose configuration	Cannot run the full stack locally	High
Backend /deployments list endpoint	Frontend dashboard has no data source	High
Backend /deployment/:id endpoint	DeploymentDetails page cannot load data	High
Backend /logs/:id endpoint	Log viewer pages have no data source	High
Nginx configuration	No reverse proxy or traffic switching	High
Actual Docker build/run in worker	Deployments are simulated, not real	High
User registration/login endpoints	User model exists but no auth routes	Medium
Project CRUD endpoints	Project model exists but no management API	Medium
Rollback mechanism	No logic to revert to previous container	Medium
WebSocket real-time updates	Polling is functional but WebSockets planned	Low
Kubernetes integration	Roadmap item	Low

17. Roadmap & Future Enhancements

The DeployMate roadmap is structured into three phases: immediate critical fixes required before any production use, near-term feature completion to deliver the core value proposition, and long-term enhancements for scale and advanced deployment strategies.

17.1 Phase 1 — Critical Fixes (Sprint 1, ~2 weeks)

These items must be completed before the platform can be used safely:

Task	Description	Owner
Fix module system inconsistency	Convert api/ to ES Modules or CommonJS consistently	Backend
Implement webhook HMAC validation	Validate X-Hub-Signature-256 on all webhook requests	Backend
Add JWT auth middleware	Protect all non-health API routes with JWT verification	Backend
Implement password hashing	Use bcrypt on user registration and login	Backend
Remove debug Redis code	Delete hardcoded user:1 set from backend/index.js	Backend
Add missing API endpoints	Implement /deployments, /deployment/:id, /logs/:id	Backend
Create Docker Compose config	Define all services for local development	DevOps

17.2 Phase 2 — Core Feature Completion (Sprint 2-3, ~4 weeks)

Task	Description	Complexity
Real webhook processing	Parse GitHub payload, create Deployment, push to BullMQ queue	Medium
Real Docker build in worker	Execute docker build and docker run commands in worker stages	High
Health check implementation	HTTP polling with retry logic and configurable timeout	Medium
Nginx configuration	Reverse proxy config with blue/green upstream switching	High
Rollback mechanism	Track previous container ID and revert on health check failure	High
User auth endpoints	POST /auth/register and POST /auth/login with JWT response	Medium
Project management API	CRUD endpoints for Project documents	Medium
Complete Docker Compose	Full multi-service stack with volumes and networking	Medium
Complete CI/CD workflows	Add all steps to deploy.yaml; remove duplicate release.yml	Low

17.3 Phase 3 — Advanced Features (Sprint 4+)

Feature	Description	Value
WebSocket real-time updates	Replace polling with server-sent events or WebSockets	High UX improvement
Blue-Green deployment strategy	Maintain two full environments; instant traffic switch	Zero-risk deployments
Multi-service architecture	Support deploying multiple services per project	Enterprise readiness
Horizontal scaling	Multiple worker instances with BullMQ concurrency control	Performance at scale
Real-time metrics dashboard	CPU, memory, request rate charts for deployed containers	Operational visibility
Kubernetes integration	Replace Docker Compose with K8s manifests for production	Cloud-native scalability
Deployment scheduling	Schedule deployments for off-peak hours using BullMQ delayed jobs	Operational flexibility
Multi-environment support	Separate staging and production deployment pipelines	Release safety
Slack/Teams notifications	Webhook notifications on deployment success or failure	Team awareness
Audit log UI	Searchable, filterable deployment history with log export	Compliance and debugging

18. Conclusion

DeployMate represents a well-conceived and architecturally sound deployment orchestration platform. The project demonstrates a strong understanding of modern software engineering principles, including event-driven architecture, separation of concerns, asynchronous job processing, and real-time user interfaces.

18.1 Architectural Strengths

The most impressive aspects of the DeployMate codebase are:

- Formal State Machine: The deployment state machine in deploymentState.js is a production-quality implementation that enforces valid lifecycle transitions and provides clear error messages for invalid operations. This level of rigour is rarely seen in early-stage projects.
- usePolling Hook Engineering: The custom React polling hook demonstrates sophisticated understanding of React's rendering model, using useRef to stabilise the fetch function reference and preventing loading flashes during background polls. This is a non-trivial implementation that solves real UX problems.
- Clean Architecture: The consistent separation of routes, controllers, and services across the backend follows established patterns that make the codebase maintainable and testable.
- Design System: The CSS custom property-based design system provides a cohesive, professional dark-themed UI without the overhead of a CSS framework.
- BullMQ Worker Design: The factory function pattern for the worker, accepting connection config as a parameter, demonstrates awareness of testability and dependency injection principles.

18.2 Areas for Improvement

The primary areas requiring attention before production readiness are:

- Security: The three critical security gaps (missing auth, no webhook validation, no password hashing) must be addressed as the highest priority. A deployment platform with unauthenticated endpoints is a significant security risk.
- Backend-Frontend Contract: Several frontend pages poll API endpoints that do not yet exist in the backend. Completing these endpoints is essential for the dashboard to display real data.
- Infrastructure Configuration: The Docker Compose and Nginx configurations are empty. Without these, the platform cannot be run as an integrated system.
- Module System Consistency: The mixed use of ES Modules and CommonJS across the api/ directory will cause runtime errors and must be resolved.

18.3 Overall Assessment

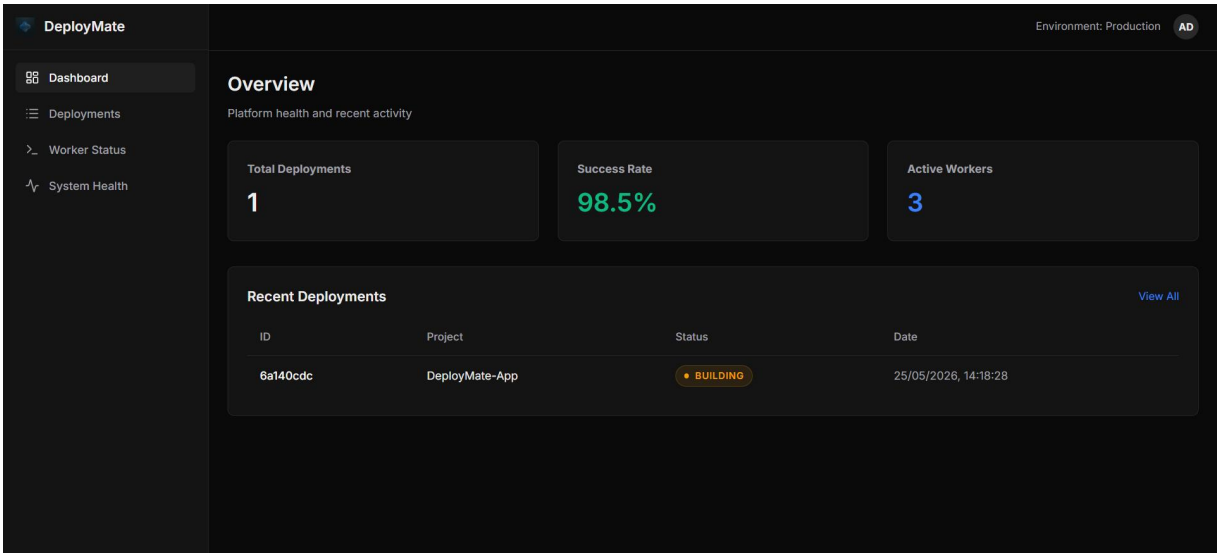
DeployMate is approximately 40% complete as a production-ready platform. The foundation — architecture, data models, state machine, frontend, and worker scaffold — is solid and well-designed. The remaining work is primarily in connecting the pieces: implementing the actual Docker operations in the worker, completing the backend API endpoints, adding authentication, and writing the infrastructure configuration.

With focused development effort across the three phases outlined in the roadmap, DeployMate has the potential to become a genuinely useful, self-hosted deployment orchestration platform that competes with commercial alternatives for teams that require on-premise control over their deployment pipeline.

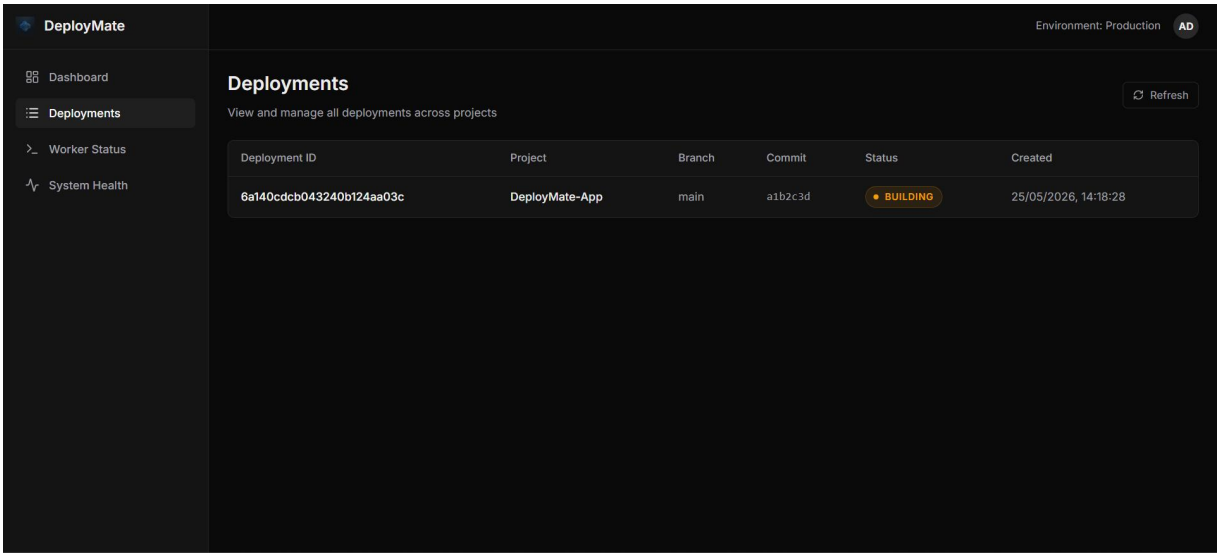
The project serves as an excellent demonstration of full-stack engineering competency, covering frontend React development, backend Node.js API design, database modelling, asynchronous job processing, and DevOps infrastructure — all within a single, coherent system.

Screenshots of the Website

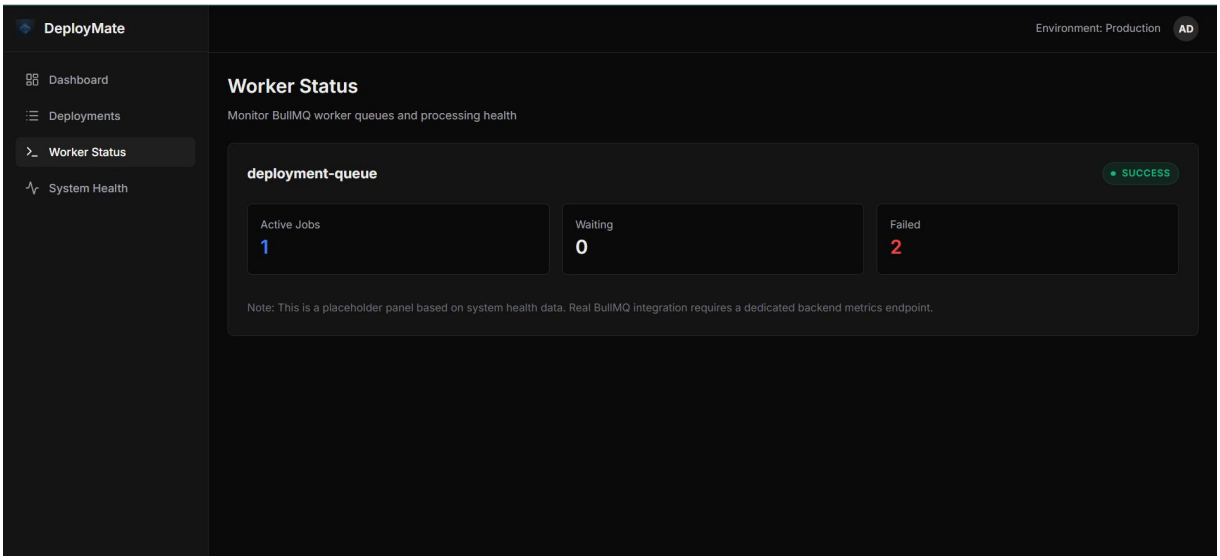
Dashboard



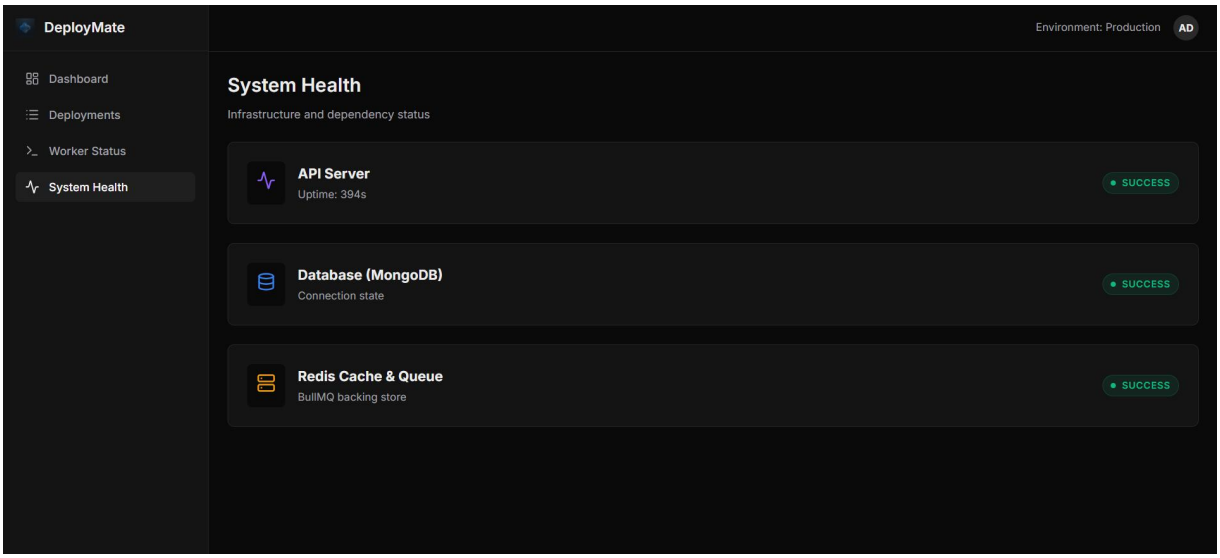
Deploymens



Worker Status



System Health



— End of Report —